Design and FPGA Implementation of a 1D CNN–LSTM Accelerator for Real-Time Human Activity Recognition

Anubhav (ROLL_NO)
National Institute of Technology, Tiruchirappalli

October 18, 2026

Abstract

This project presents the design, hardware translation and resource characterisation of a compact 1D convolutional–recurrent neural network for human activity recognition from inertial sensor data, targeting a Xilinx Zynq UltraScale+ MPSoC. A 1D CNN front end followed by an LSTM classifier is trained on the UCI HAR dataset and reaches 90.91% test accuracy using only 4,374 parameters across six sensor channels. The trained network is converted to synthesisable C++ using `hls4ml`, compiled to register-transfer level by Vitis HLS and packaged as an AXI4-Stream IP core. Post-synthesis characterisation on the XCZU7EV device reports 33,500 LUT (14% of the part), 54 DSP slices (3%), 159 block RAMs (25%) and an inference latency of 1.60 ms at 100 MHz, against a 2.56 s input window — a margin of three orders of magnitude. The work additionally quantifies how high-level synthesis configuration, rather than network size, dominates hardware cost: a single change of activation layer construct reduced the two activation blocks from 222,550 LUT to 2,182 LUT, and selecting a streaming interface over a parallel one reduced the complete design from approximately 298,000 LUT to 35,745 LUT. The quantization strategy is documented in full, including a derivation of `hls4ml`'s automatic accumulator widening that reproduces all four generated accumulator types exactly. A negative result is also reported: the design does not fit a low-cost Gowin GW1NR-9 device, because the LSTM's sigmoid and tanh lookup tables alone require more block RAM than that device contains. Results in this report are high-level-synthesis estimates; bitstream generation and on-board deployment remain future work.

Contents

1 Introduction **2**

2 Related Work **2**
2.1 Deep learning for sensor-based activity recognition 2
2.2 The UCI HAR benchmark 2
2.3 Neural network inference on FPGAs 3
2.4 Joint-embedding predictive architectures 3

3 Theoretical Background **3**
3.1 Inertial sensing and windowing 3
3.2 1D convolution for time series 3
3.3 Long short-term memory 4
3.4 Batch normalisation and its folding 4
3.5 Fixed-point arithmetic 4
3.6 Reuse factor 4

4 System Description **4**
4.1 Overall architecture . . . 4
4.2 Network architecture . . . 5
4.3 Channel selection . . . 5

5 Quantization Strategy **6**
5.1 Uniform fixed-point representation . . . 6
5.2 Rounding and overflow behaviour . . . 6
5.3 Automatic accumulator widening . . . 7
5.4 Quantization-aware alternatives . . . 7

6 Design and Implementation **8**
6.1 Toolflow . . . 8
6.2 Framework constraint: PyTorch could not be used . . . 9
6.3 Streaming versus parallel interfaces . . . 9
6.4 Activation layer construction . . . 9
6.5 Generated IP interface . . . 9

7 Results **10**
7.1 Classification accuracy . . . 10
7.2 Interface model . . . 10
7.3 Target device comparison . . . 10
7.4 Final design on the target device . . . 11
7.5 Arithmetic precision . . . 11
7.6 Activation lookup tables . . . 11
7.7 Network width and the memory ceiling . . . 11
7.8 Negative result: infeasibility on a small FPGA . . . 12

8 Applications and Advantages **12**

9 Limitations and Future Scope **12**
9.1 Limitations . . . 12
9.2 Future scope . . . 13

10 Conclusion **13**

11 References **14**

A Development Environment **15**

B Target Device Resources **15**

C Reproduction **15**

D Dataset Summary **15**

1 Introduction

Human activity recognition (HAR) from body-worn inertial sensors underpins applications in healthcare monitoring, fall detection and personal safety devices. Such systems are typically battery-powered and latency-sensitive, which motivates performing inference at the edge rather than transmitting raw sensor streams to a server.

Field-programmable gate arrays (FPGAs) are attractive for this role. Unlike a general-purpose processor, an FPGA executes a neural network as dedicated parallel hardware, and unlike a fixed-function ASIC it can be reprogrammed as the model evolves. The difficulty is that translating a trained network into efficient hardware is not automatic: the same network can differ in area by two orders of magnitude depending on how it is mapped.

This project builds the complete translation chain — train, convert, synthesise, package — for a 1D CNN–LSTM activity classifier, and measures what each stage actually costs in hardware. The specific contributions are:

1. A compact CNN–LSTM achieving 90.91% six-class accuracy on UCI HAR using 4,374 parameters and only the six sensor channels an MPU6050 physically provides.
2. A complete, reproducible `hls4ml` → Vitis HLS → Vivado IP flow, with the generated core's interface characterised at the port level.
3. Quantitative resource characterisation across interface type, arithmetic precision, activation-table size and network width, isolating which choices matter and which do not.
4. A documented negative result establishing why gated recurrence is infeasible on a small (<10k LUT) FPGA, with the block RAM accounting that explains it.

2 Related Work

2.1 Deep learning for sensor-based activity recognition

The combination of convolutional feature extraction with recurrent temporal modelling is the established architecture for wearable HAR. Ordóñez and Roggen [2] introduced DeepConvLSTM, stacking 1D convolutional layers beneath LSTM layers, and demonstrated that the convolutional front end learns local motion primitives while the recurrent stage captures their temporal ordering. Hammerla et al. [3] systematically compared deep, convolutional and recurrent models across wearable HAR benchmarks and found recurrent architectures particularly effective for the naturalistic, variable-duration activities typical of this domain.

The architecture used here follows that lineage directly, but at drastically reduced width. Where DeepConvLSTM employs 64 convolutional filters and 128 hidden units, this work targets 16 of each, because the parameter budget is set by on-chip memory rather than by accuracy alone.

2.2 The UCI HAR benchmark

Anguita et al. [1] published the UCI HAR dataset, comprising recordings from 30 participants performing six activities with a waist-mounted smartphone sampled at 50 Hz. Its wide adoption provides published baselines for comparison, which is the reason it was selected here over sensor-matched alternatives. Its principal limitation for this project is discussed in Section 9: it contains no fall or distress class.

2.3 Neural network inference on FPGAs

Two toolflows dominate academic FPGA inference. FINN, introduced by Umuroglu et al. [5], targets heavily quantised (binarised) networks and generates streaming dataflow architectures. `hls4ml`, introduced by Duarte et al. [4] for microsecond-latency inference in particle physics triggers, converts Keras and PyTorch models into Vivado/Vitis HLS C++ and exposes per-layer control over fixed-point precision and multiplier reuse.

This project uses `hls4ml` because its reuse-factor abstraction maps cleanly onto the design question here: the available latency budget (2.56 s) is enormous relative to the computation, so the appropriate strategy is to trade all of that latency for reduced area. Section 7 shows this trade is worth roughly an order of magnitude.

2.4 Joint-embedding predictive architectures

The project was initially scoped around a Transformer-based joint-embedding predictive architecture (JEPA), following LeCun's proposal [8] and the image-domain realisation of Assran et al. [9]. That direction was abandoned early: self-attention and layer normalisation are expensive to synthesise, and the self-supervised pre-training that gives JEPA its advantage requires far more unlabelled data than this project could collect. The supervised CNN–LSTM was adopted instead. The project name retains the original acronym for continuity with earlier documentation.

3 Theoretical Background

3.1 Inertial sensing and windowing

A six-axis inertial measurement unit reports linear acceleration along three orthogonal axes and angular velocity about those same axes. Activity is not identifiable from an instantaneous sample; it is a property of a time interval. The signal is therefore segmented into fixed-length windows. At the UCI HAR sampling rate of 50 Hz, the 128-sample window used here spans

$$T_{\text{window}} = \frac{128 \text{ samples}}{50 \text{ Hz}} = 2.56 \text{ s}, \tag{1}$$

long enough to contain several gait cycles. This figure sets the real-time constraint: inference must complete within 2.56 s to keep pace with the sensor.

3.2 1D convolution for time series

A 1D convolutional layer slides F learned kernels of length k along the time axis, each spanning all C input channels. With no padding, an input of T time steps produces

$$T_{\text{out}} = T - k + 1 \tag{2}$$

output steps and requires

$$\text{MACs} = T_{\text{out}} \times k \times C \times F \tag{3}$$

multiply–accumulate operations. This expression matters for hardware: the MAC count, not the parameter count, determines arithmetic cost, and the factor T_{out} is precisely what the streaming interface of Section 6.3 avoids unrolling.

3.3 Long short-term memory

An LSTM cell [6] maintains a cell state c_t regulated by three gates. For input x_t and previous hidden state h_{t-1}:

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i) \tag{4}$$
$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f) \tag{5}$$
$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o) \tag{6}$$
$$\tilde{c}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c) \tag{7}$$
$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t \tag{8}$$
$$h_t = o_t \odot \tanh(c_t) \tag{9}$$

Two consequences are central to this report. First, the gating requires both a sigmoid and a hyperbolic tangent, neither of which is cheap in fixed-point hardware — both are realised as lookup tables. Second, there are four weight matrices of each kind, so a cell with H hidden units over n inputs holds $4H(n+H)$ weights and evaluates $4H$ nonlinearities per time step. The hardware consequence of that factor of four is measured in Section 7.6.

3.4 Batch normalisation and its folding

Batch normalisation [7] applies, at inference time, a fixed affine transform per channel:

$$y = \gamma \cdot \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta. \tag{10}$$

Because this is affine and immediately follows a convolution, which is also affine, the two can be composed into a single convolution with modified weights and a bias. The synthesis tool performs this automatically, so batch normalisation costs nothing in hardware. The convolutions in this design are therefore declared without bias terms, since the folded β supplies them.

3.5 Fixed-point arithmetic

FPGAs implement floating-point arithmetic only at considerable cost, so inference is performed in fixed point. The notation `ap_fixed<W,I>` denotes a W-bit signed value with I integer bits and $W - I$ fractional bits, giving a resolution of $2^{-(W-I)}$ and a range of $\pm 2^{I-1}$. This design uses `ap_fixed<16,6>` throughout. The representation, its rounding and overflow modes, and the automatic widening of intermediate accumulators are treated in full in Section 5.

3.6 Reuse factor

The reuse factor R specifies how many times a single hardware multiplier is re-used within a layer. For a layer of N MACs, the synthesised design instantiates N/R multipliers and takes R cycles. Setting $R = N$ yields one multiplier operating fully sequentially — minimum area, maximum latency. Given the 2.56 s budget, this is the correct operating point for every layer in this design.

4 System Description

4.1 Overall architecture

```
MPU6050 (6-axis IMU)                    [not yet integrated]
         |
```

```
          v
Window buffer: 128 samples x 6 channels
          |
          v
+--------------------------------------+
|  CNN-LSTM accelerator (FPGA fabric)  |
|  AXI4-Stream in  : 96 bit  x 128 beats|
|  AXI4-Stream out : 384 bit x 1 beat  |
+--------------------------------------+
          |
          v
argmax -> class -> alert action          [not yet implemented]
```

4.2 Network architecture

Layer	Configuration	Output shape	Params
Input	128 steps × 6 channels	(128, 6)	0
Conv1D	16 filters, $k = 5$, no bias	(124, 16)	480
BatchNorm + ReLU	folded into the convolution	(124, 16)	64
Conv1D	16 filters, $k = 5$, no bias	(120, 16)	1,280
BatchNorm + ReLU	folded into the convolution	(120, 16)	64
MaxPooling1D	pool 2	(60, 16)	0
Dropout	$p = 0.3$, training only	(60, 16)	0
LSTM	16 hidden, final state only	(16,)	2,176
Dropout	$p = 0.3$, training only	(16,)	0
Dense + ReLU	16 units	(16,)	272
Dense	6 units, logits	(6,)	102
		Total trainable parameters	**4,374**

Table 1: Deployed network. Shapes are those reported by the synthesis tool after batch-normalisation folding and dropout removal.

The output is six raw logits. The softmax normally applied in training is omitted in hardware, because it is monotonic and the decision is taken by `argmax`, which the normalisation does not affect.

4.3 Channel selection

The UCI HAR dataset provides nine channels: body acceleration, body angular velocity and total acceleration, each in three axes. This design uses six — total acceleration and body angular velocity — because those are exactly what an MPU6050 outputs at its register interface. Body acceleration is total acceleration with the gravity component removed by a Butterworth filter applied offline by the dataset authors; using it would require reimplementing that filter in hardware ahead of the network. Discarding it removes that burden entirely at negligible accuracy cost.

For the same reason no input normalisation is applied. The network is trained on raw physical units, so on-device preprocessing reduces to a constant scaling of the sensor's integer output — a shift rather than a per-channel affine transform.

5 Quantization Strategy

Neural networks are trained in 32-bit floating point, but floating-point arithmetic is expensive to synthesise. Deployment therefore requires mapping every weight, activation and intermediate result onto a fixed-point representation. This section states what that mapping is in this design, what the synthesis tool derives automatically, and what the alternatives would be.

The quantization used here is post-training and uniform. The network is trained entirely in floating point and the trained weights are then mapped to a fixed-point format specified in the `hls4ml` configuration. No quantization-aware training was performed. The consequences of that choice, and the techniques that would improve on it, are discussed in Section 5.4.

5.1 Uniform fixed-point representation

A value in the format `ap_fixed<W,I>` is a W-bit two's-complement integer interpreted with I bits before the binary point, leaving $f = W - I$ fractional bits. The representable set is the uniform grid

$$\mathcal{Q}_{W,I} = \left\{ k \cdot \Delta \;\middle|\; k \in \mathbb{Z},\ -2^{W-1} \le k \le 2^{W-1} - 1 \right\}, \qquad \Delta = 2^{-f}, \tag{11}$$

so the quantization step and the dynamic range are set independently by f and I respectively:

$$\Delta = 2^{I-W}, \qquad x \in \left[-2^{\,I-1},\ 2^{\,I-1} - \Delta \right]. \tag{12}$$

This design uses `ap_fixed<16,6>` throughout, giving

$$\Delta = 2^{-10} \approx 9.77 \times 10^{-4}, \qquad x \in [-32,\ 32 - 2^{-10}]. \tag{13}$$

The format was chosen from the physical units of the input rather than by search. Accelerometer readings are expressed in g and angular velocity in rad/s, both of which lie well inside ± 32 for human motion, so six integer bits provide ample headroom while ten fractional bits resolve the signal far below sensor noise.

The error introduced by quantization behaves as additive noise of variance $\Delta^2/12$ for a uniformly distributed residual, giving the standard result that the signal-to-quantization-noise ratio improves by approximately

$$\Delta\mathrm{SQNR} \approx 20 \log_{10} 2 \approx 6.02 \text{ dB} \tag{14}$$

per additional fractional bit. Moving from `ap_fixed<16,6>` to `ap_fixed<8,4>` removes six fractional bits and therefore costs roughly 36 dB of arithmetic fidelity — which is why the resource saving reported in Section 7 cannot be accepted without measuring its accuracy cost.

5.2 Rounding and overflow behaviour

A fixed-point type is incompletely specified by W and I alone: the behaviour on values falling between grid points, and on values exceeding the range, must also be fixed. Inspection of the generated project shows the default modes are in use throughout, the full type being `ap_fixed<16,6,AP_TRN,AP_WRAP,0>`.

Truncation (`AP_TRN`). The value is rounded toward $-\infty$:

$$Q(x) = \Delta \left\lfloor \frac{x}{\Delta} \right\rfloor, \qquad e = Q(x) - x \in (-\Delta,\ 0]. \tag{15}$$

The error is therefore strictly non-positive, with mean $-\Delta/2$, rather than the zero-mean error of round-to-nearest (`AP_RND`), for which $e \in (-\Delta/2, \Delta/2]$. This distinction is not cosmetic. Over a dot product of N terms the truncation bias accumulates *linearly*,

$$\mathbb{E}\left[\sum_{i=1}^{N} e_i\right] = -\frac{N\Delta}{2}, \tag{16}$$

whereas a zero-mean rounding error accumulates only as $\mathcal{O}(\sqrt{N})$ in standard deviation. Wider layers are thus systematically biased downward under truncation. For the 80-term dot products of the second convolutional layer this is a deterministic offset of $40\Delta \approx 0.039$ before the nonlinearity.

Wrapping (`AP_WRAP`). On overflow the value is reduced modulo 2^I, so a result marginally above the maximum representable value becomes the most negative value. Unlike saturation (`AP_SAT`), which clamps, wrapping converts a small numerical excursion into a maximal sign error. Saturation costs a comparator per value and was not enabled; the design instead relies on the six integer bits being generous relative to the physical signal range.

Both defaults are reasonable here given that headroom, but they are defaults rather than decisions, and they are the first things to revisit should fixed-point accuracy prove disappointing.

5.3 Automatic accumulator widening

Quantizing weights and activations to W bits does not mean the arithmetic is performed at W bits. A product of `ap_fixed<`W_x`,`I_x`>` and `ap_fixed<`W_w`,`I_w`>` operands requires $W_x + W_w$ bits to be exact, and summing N such products requires a further $\lceil \log_2 N \rceil$ integer bits to be guaranteed overflow-free. `hls4ml` sizes these accumulators automatically, and inspection of the generated type definitions shows it applies

$$W_{\text{acc}} = W_x + W_w + \lceil \log_2 N \rceil + 1, \tag{17}$$

$$I_{\text{acc}} = I_x + I_w + \lceil \log_2 N \rceil + 1, \tag{18}$$

where the additional bit is a guard bit. The rule was checked against every layer of the generated design, with $N = k \cdot C_{\text{in}}$ for the convolutions and $N = n_{\text{in}}$ for the dense layers:

Layer	N	$\lceil \log_2 N \rceil$	**Predicted**	**Generated**
`conv1d`	30	5	`ap_fixed<38,18>`	`ap_fixed<38,18>`
`conv1d_1`	80	7	`ap_fixed<40,20>`	`ap_fixed<40,20>`
`dense`	16	4	`ap_fixed<37,17>`	`ap_fixed<37,17>`
`logits`	16	4	`ap_fixed<37,17>`	`ap_fixed<37,17>`

Table 2: Predicted versus generated accumulator types, for `ap_fixed<16,6>` inputs and weights throughout. The rule reproduces all four exactly.

Two practical consequences follow. First, the accumulators are safe by construction, so overflow cannot occur *inside* a dot product. Second, the risk is instead concentrated at the layer boundaries, where the wide accumulator is cast back down to `ap_fixed<16,6>` under the wrapping mode described above. The network output is a visible instance of this: the logits emerge as `ap_fixed<37,17>` rather than the `<16,6>` used internally, because the final accumulator is never narrowed. This is harmless for classification, since `argmax` is invariant to the representation, but a host reading those values must interpret them with 17 integer bits.

5.4 Quantization-aware alternatives

Post-training quantization maps a network that was never aware it would be quantized. Quantization-aware training (QAT) instead simulates the quantizer during the forward pass, allowing the

weights to adapt to it. **Neither technique below was used in this project.** Both are the natural next step, and are described here because they bear directly on the unmeasured accuracy risk identified in Section 9.

The straight-through estimator. The obstacle common to all QAT is that the quantizer is piecewise constant, so $\partial Q/\partial x = 0$ almost everywhere and no gradient reaches the weights. The standard remedy is the straight-through estimator, which applies the quantizer in the forward pass but substitutes the identity in the backward pass:

$$\text{forward: } \hat{x} = Q(x), \qquad \text{backward: } \frac{\partial L}{\partial x} \approx \frac{\partial L}{\partial \hat{x}} \cdot \mathbf{1}[\,|x| \le x_{\max}\,]\,. \tag{19}$$

The indicator suppresses gradients for saturated values, which would otherwise be pushed further out of range.

QKeras applies this at tensor granularity. Each layer is given a quantizer such as `quantized_bits(`b`,` i`,` α`)`, fixing one bitwidth for an entire weight tensor, and the network is fine-tuned from the floating-point checkpoint. The bitwidths remain hyperparameters chosen by the designer.

High Granularity Quantization (HGQ) [10] removes that manual choice. Rather than one bitwidth per tensor, HGQ assigns a bitwidth b_i to each individual parameter and learns it jointly with the weights by gradient descent. For this to be meaningful the hardware cost must enter the objective, so the training loss is augmented with a differentiable resource proxy. The natural proxy is the bit-operation count: a multiplication of a b_w-bit weight by a b_a-bit activation costs approximately $b_w \cdot b_a$ bit-operations, so for a network with multiplication set $\mathcal{M}$,

$$\text{EBOPs}(\mathbf{b}) \approx \sum_{(w,a)\in\mathcal{M}} b_w \cdot b_a, \qquad \mathcal{L} = \mathcal{L}_{\text{task}} + \beta \cdot \text{EBOPs}(\mathbf{b}). \tag{20}$$

The bitwidths are held continuous during optimisation and discretised for export, with the straight-through estimator supplying gradients through the quantizer. The coefficient β traces a Pareto frontier between accuracy and hardware cost, so instead of selecting a single uniform format the designer selects an operating point on a measured curve.

This is precisely the choice that was made by hand in this project, and made crudely: `ap_fixed<16,6>` was applied uniformly to every layer, even though the layers differ substantially in dynamic range, and the 8-bit alternative was evaluated only as a second uniform format. HGQ would allow the first convolution, which sees bounded physical units, to use far fewer bits than the LSTM gates, which do not. Neither library is installed in the current environment, so adopting either requires toolchain setup in addition to retraining.

6 Design and Implementation

6.1 Toolflow

```
Keras / TensorFlow  --->  hls4ml  --->  Vitis HLS  --->  Vivado  ---> FPGA
 train, 90.91%          C++ + tcl        RTL,           IP, block    bitstream
                                         csynth         design
```

Training uses TensorFlow 2.21 with Adam at a learning rate of 10^{-3}, batch size 64, for 40 epochs, with sparse categorical cross-entropy computed from logits. The dataset split is the one published with UCI HAR: 7,352 training and 2,947 test windows, partitioned by subject so that no participant appears in both.

6.2 Framework constraint: PyTorch could not be used

The network was first implemented in PyTorch. It could not be converted. The `hls4ml` PyTorch front end (version 1.3.0) provides no handler for the `transpose` operation, and a PyTorch CNN–LSTM requires two of them: `nn.Conv1d` expects channel-first tensors while `nn.LSTM` expects (batch, time, feature) ordering. Three alternative tensor layouts were attempted; all failed with `Unsupported function transpose`.

Keras `Conv1D` is channels-last by construction, so the identical architecture converts with no transpose operations at all. The recurrent layer itself is not the obstacle — the PyTorch LSTM handler is present and functional — which makes this purely an artefact of tensor-layout convention. The implementation was therefore moved to Keras.

6.3 Streaming versus parallel interfaces

`hls4ml` offers two interface models. Under `io_parallel` a layer receives its entire input tensor at once and is unrolled across all output positions. Under `io_stream` the layer receives one time step at a time through a FIFO and maintains a line buffer, so the hardware instantiated scales with filter count rather than with window length.

For a convolutional layer producing 124 output positions this distinction is decisive, and it is not something the reuse factor can compensate for: reuse shares multipliers *within* a dot product but does not prevent unrolling *across* output positions. The measured effect is reported in Section 7.

6.4 Activation layer construction

Keras offers two syntactically equivalent ways to apply a rectified linear unit: the `ReLU` layer class and `Activation('relu')`. They compute identical functions, since the `ReLU` layer's threshold defaults to zero. They do not synthesise identically. `hls4ml` maps the layer class to a *parameterised* thresholded-ReLU primitive that carries a runtime threshold comparison for every element, while `Activation('relu')` maps to a plain ReLU. Across the 3,904 activations of the convolutional front end the difference is substantial, and is quantified in Table 5.

6.5 Generated IP interface

Vitis HLS exports the design as an IP-XACT package for Vivado. The top-level interface is:

Port	Width	Description
`imu_window`	96 bit	AXI4-Stream slave; 6 channels × `ap_fixed<16,6>`, 128 beats
`layer14_out`	384 bit	AXI4-Stream master; 6 logits, 1 beat
`ap_clk`, `ap_rst_n`	1 bit	clock and active-low reset
`ap_start`, `ap_done`	1 bit	block-level handshake
`ap_ready`, `ap_idle`	1 bit	block-level handshake

Table 3: Top-level ports of the exported IP core.

Three properties of this interface constrain system integration. Control is a block-level handshake rather than an AXI4-Lite register map, so a processor cannot start the core by writing a register; for a streaming kernel `ap_start` is instead tied asserted. The streams carry no `TLAST` side-channel, which a DMA engine normally uses to delimit packets. Finally, 96 and 384 bits are not among the stream widths an AXI DMA supports; both are integer multiples of 32 bits, so width converters are required.

7 Results

All figures in this section are Vitis HLS 2023.2 post-synthesis estimates for `xczu7ev-ffvc1156-2-e` unless stated otherwise. Estimates of this kind are generally pessimistic for LUT count; place-and-route figures are expected to be lower.

7.1 Classification accuracy

Configuration	Parameters	Test accuracy
16 filters / 16 hidden / 16 dense	4,374	90.91%
64 filters / 64 hidden / 64 dense	60,486	92.09%

Table 4: Accuracy against network width on the UCI HAR test split.

A 13.8-fold increase in parameters yields 1.18 percentage points of accuracy. The smaller network is retained for the reason given in Section 7.7.

7.2 Interface model

Block	`io_parallel` LUT	`io_stream` LUT
Conv1D #1	21,729	2,911
Activation #1	113,099	1,075
Conv1D #2	53,751	4,993
Activation #2	109,451	1,107
MaxPooling	35	1,141
Complete design	**≈298,000**	**35,745**

Table 5: Effect of interface model. The `io_parallel` column is the convolutional front end only; the recurrent and dense layers had not been reported when that synthesis was halted.

Two effects are superimposed in this table. The streaming interface removes the unrolling across output positions. Separately, the activation figures of 113,099 and 109,451 LUT are an artefact of the thresholded-ReLU construct described above; replacing it with `Activation('relu')` reduced those two blocks to 1,075 and 1,107 LUT — a combined reduction from 222,550 to 2,182 LUT, by a factor of 102, with no change whatsoever to the computed function or to the 90.91% accuracy.

7.3 Target device comparison

Device	LUT	DSP	BRAM	Outcome
XCZU7EV (ZCU104)	16%	3%	25%	Fits
XC7Z020 (Zynq-7020)	67%	24%	56%	Would fit
GW1NR-9 (Tang Nano 9K)	414%	270%	612%	Does not fit

Table 6: Utilisation of the same 35,745 LUT design across three candidate devices.

Resource	Used	Utilisation
LUT	33,500	14%
Flip-flops	21,724	4%
DSP slices	54	3%
Block RAM (18 Kb)	159	25%
UltraRAM	0	0%
Clock	100 MHz, timing met (0.37 ns slack)	
Latency	160,339 cycles = 1.60 ms	
Throughput margin	2.56 s window / 1.60 ms = 1600×	

Table 7: Final configuration: `io_stream`, maximum reuse, `ap_fixed<16,6>`, on XCZU7EV at 100 MHz.

7.4 Final design on the target device

7.5 Arithmetic precision

Precision	LUT	DSP	BRAM
`ap_fixed<16,6>`	35,745	54	159
`ap_fixed<8,4>`	26,526	16	109

Table 8: Effect of halving the word length (50 MHz runs).

Halving the word length reduces LUT usage by 26%, block RAM by 31% and DSP usage by 70%. The accuracy cost of this reduction has not been measured, and 16-bit precision was retained because the target device has ample margin.

7.6 Activation lookup tables

The LSTM's sigmoid and tanh evaluations are realised as lookup tables, and these dominate the recurrent layer's memory footprint. Two experiments isolate what drives their size.

Table	$H = 16$	$H = 64$
sigmoid	24 BRAM	96 BRAM
tanh	8 BRAM	32 BRAM
Total	32 BRAM	128 BRAM

Table 9: Activation-table memory against LSTM hidden size.

Reducing the configured table depth from 1024 to 256 entries changed nothing: the generated constant was verified to be 256 in the synthesised project, and the reported resources were byte-identical. Table footprint therefore does not scale with depth. It scales with hidden size, exactly linearly — a factor of four in H producing a factor of four in memory — which is consistent with the table being replicated to permit parallel access across the $4H$ gate evaluations each time step requires.

7.7 Network width and the memory ceiling

The wider network is comfortable in logic and arithmetic but exceeds available block RAM by 4%. Since the device's 96 UltraRAM blocks (27 Mb) remain entirely unused, this is a mapping

Configuration	LUT	DSP	BRAM	Latency
16/16/16	33,500 (14%)	54 (3%)	159 (25%)	1.60 ms
64/64/64	126,167 (54%)	198 (11%)	**654 (104%)**	25.4 ms

Table 10: The wider network exceeds the device's block RAM.

problem rather than a capacity one, and directing the weight arrays to UltraRAM is the obvious remedy. That work was not completed, so the narrower network remains the only configuration measured to fit, and the 1.18-point accuracy difference stands as its cost.

7.8 Negative result: infeasibility on a small FPGA

The project initially targeted a Sipeed Tang Nano 9K (Gowin GW1NR-9: 8,640 LUT4, 20 DSP, 26 block RAMs). The design does not fit, and the reason is structural rather than a question of tuning.

The LSTM accounts for 56% of all logic in the design. Within it, the sigmoid and tanh tables alone require 32 block RAMs — more than the 26 the entire device contains — before a single weight or stream buffer is allocated. Since gated recurrence requires both nonlinearities by definition, no reduction in precision, table depth or weight count removes this floor. Measurements confirmed this: 8-bit precision brought DSP usage within budget (16 of 20) but left LUT usage 3.1× and block RAM 4.2× over.

The practical conclusion is that an LSTM is the wrong primitive for a sub-10k-LUT device. A purely convolutional network with global pooling has no such tables; the measured convolutional front end occupies 7,904 LUT and would fit that device with room remaining for peripheral logic.

8 Applications and Advantages

- Real-time activity classification with a 1600× latency margin, leaving substantial headroom for additional processing stages.
- Inference performed locally, so raw inertial data need never leave the device — relevant where the sensor stream is privacy-sensitive.
- A reproducible toolflow in which retraining the network and regenerating hardware requires no manual RTL modification.
- Deterministic latency, as the design contains no cache, operating system or contended shared resource.
- A characterised design space, allowing the accuracy/area trade-off to be re-evaluated quickly if the target device changes.

9 Limitations and Future Scope

9.1 Limitations

- **No hardware deployment.** All results are high-level-synthesis estimates. No bitstream has been generated and the design has not been executed on a physical device. Place-and-route figures will differ.
- **Fixed-point accuracy unverified.** The 90.91% figure is the floating-point model's. The accuracy of the 16-bit fixed-point implementation has not been measured. Since the

quantization is post-training and uniform, with no quantization-aware fine-tuning, there is no guarantee the mapping is benign.

- **The sensor is not integrated.** The MPU6050 interface, window assembly and alert logic are specified but unimplemented.
- **The task is activity recognition, not safety detection.** UCI HAR contains six ordinary activities and no fall, impact or distress class. The trained network cannot detect events it has never observed, and no claim of fall or assault detection is supported by these results.
- **Window-level, not event-level.** Classification is performed on 2.56 s windows in isolation, with no temporal smoothing or hysteresis across consecutive windows.

9.2 Future scope

- Generate the bitstream and validate on hardware, comparing place-and-route utilisation against the estimates reported here.
- Measure fixed-point accuracy using bit-accurate C simulation, and re-evaluate 8-bit precision against the measured loss.
- Replace uniform post-training quantization with quantization-aware training. High Granularity Quantization (Section 5.4) is the strongest candidate, since it learns a per-parameter bitwidth against a differentiable resource cost rather than requiring one format to be chosen by hand for the whole network.
- Map weight arrays to UltraRAM to accommodate the wider network and recover the 1.18-point accuracy difference.
- Retrain on a fall-annotated dataset (for instance EdgeFall) to support the safety application the architecture was originally intended for.
- Integrate the MPU6050 and implement continuous windowed inference with temporal smoothing.
- Measure power consumption, which is the metric that would justify an FPGA over a microcontroller for a battery-powered device, and which this work does not address.

10 Conclusion

A 1D CNN–LSTM activity classifier was trained to 90.91% accuracy on UCI HAR using 4,374 parameters, converted to synthesisable hardware through `hls4ml` and Vitis HLS, and packaged as an AXI4-Stream IP core. On the XCZU7EV the design occupies 14% of available logic and completes inference in 1.60 ms against a 2.56 s window.

The more transferable finding concerns how little of the hardware cost was determined by the network itself. Two configuration choices — the activation layer construct and the interface model — together accounted for roughly an order of magnitude in area, while leaving the computed function unchanged. The corresponding negative result is equally concrete: the sigmoid and tanh tables an LSTM requires exceed the entire block RAM capacity of a small FPGA independently of precision or table depth, which rules out gated recurrence on such devices rather than merely making it difficult.

The flow from trained network to packaged IP is complete and reproducible. What remains is the hardware half: bitstream generation, on-board validation, sensor integration, and retraining on data containing the events a safety device would need to recognise.

11 References

References

[1] Anguita, D., Ghio, A., Oneto, L., Parra, X., & Reyes-Ortiz, J. L. (2013). *A Public Domain Dataset for Human Activity Recognition Using Smartphones.* 21st European Symposium on Artificial Neural Networks (ESANN).

[2] Ordóñez, F. J., & Roggen, D. (2016). *Deep Convolutional and LSTM Recurrent Neural Networks for Multimodal Wearable Activity Recognition.* Sensors, 16(1), 115.

[3] Hammerla, N. Y., Halloran, S., & Plötz, T. (2016). *Deep, Convolutional, and Recurrent Models for Human Activity Recognition using Wearables.* Proceedings of IJCAI 2016.

[4] Duarte, J., et al. (2018). *Fast inference of deep neural networks in FPGAs for particle physics.* Journal of Instrumentation, 13, P07027.

[5] Umuroglu, Y., Fraser, N. J., Gambardella, G., Blott, M., Leong, P., Jahre, M., & Vissers, K. (2017). *FINN: A Framework for Fast, Scalable Binarized Neural Network Inference.* Proceedings of FPGA 2017.

[6] Hochreiter, S., & Schmidhuber, J. (1997). *Long Short-Term Memory.* Neural Computation, 9(8), 1735–1780.

[7] Ioffe, S., & Szegedy, C. (2015). *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift.* Proceedings of ICML 2015.

[8] LeCun, Y. (2022). *A Path Towards Autonomous Machine Intelligence.* OpenReview.

[9] Assran, M., et al. (2023). *Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture.* Proceedings of CVPR 2023.

[10] Sun, C., Årrestad, T. K., Loncar, V., Ngadiuba, J., & Spiropulu, M. (2024). *Gradient-based Automatic Mixed Precision Quantization for Neural Networks On-Chip.* (High Granularity Quantization, HGQ.)

[11] AMD/Xilinx. *Vitis High-Level Synthesis User Guide* (UG1399), version 2023.2.

[12] AMD/Xilinx. *ZCU104 Evaluation Board User Guide* (UG1267).

A Development Environment

Component	Version	Role
Python	3.11.9	host environment
TensorFlow / Keras	2.21 / 3.14	network training
`hls4ml`	1.3.0	network → HLS C++
Vitis HLS	2023.2	C++ → RTL, IP packaging
Vivado	2023.2	block design, implementation
Target device	XCZU7EV-2FFVC1156E (ZCU104)	

B Target Device Resources

Resource	XCZU7EV	GW1NR-9
LUT	230,400	8,640
Flip-flops	460,800	—
DSP slices	1,728	20
Block RAM	11.0 Mb	468 Kb
UltraRAM	27.0 Mb	—
Processor	4×Cortex-A53	none

C Reproduction

```
# Build the dataset cache and train the network
python -m jepasafe.demopart --epochs 30
python -m jepasafe.cnnlstm_keras --epochs 40

# Convert to an HLS project
python -m jepasafe.export_hls

# C synthesis
export PATH="/c/Xilinx/Vitis_HLS/2023.2/bin:$PATH"
cd hls/jepasafe && vitis_hls -f build_prj.tcl
# report: jepasafe_prj/solution1/syn/report/csynth.rpt

# Package as a Vivado IP
cd hls/jepasafe && vitis_hls -f ../../scripts/export_ip.tcl
# output: jepasafe_prj/solution1/impl/ip/nitt_hls_jepasafe_1_0.zip
```

D Dataset Summary

Source	UCI HAR (Anguita et al., 2013)
Participants	30, split by subject between train and test
Sampling rate	50 Hz
Window	128 samples (2.56 s)
Training windows	7,352
Test windows	2,947
Channels used	6 of 9 (total acceleration, body angular velocity)
Classes	walking, upstairs, downstairs, sitting, standing, laying